

PROJECT REPORT-DIGITAL FORENSICS

Comparative study on MySQL and NoSQL Injection Detection

TEAM:

SHREEVARSHA S [PES1UG23CS555] (MYSQL PART)

SHIVANI J [PES1UG23CS549] (NOSQL PART)

SUPERVISOR'S NAME: DR. SAPNA V M

UNIVERSITY: PES UNIVERSITY, BENGALURU

INTERNSHIP DURATION : JUNE 2025-JULY 2025

DATE OF SUBMISSION: 18TH JULY 2025

ACKNOWLEDGMENT

I would like to express my sincere gratitude to Sapna Ma'am ,Teja Ma'am and Pooja Ma'am for their constant support and guidance during my internship. I also thank the Digital Forensics Lab team for providing me with valuable insights into practical cybersecurity. Special thanks to my family and peers for their motivation and encouragement throughout this project.

ABSTRACT

This report summarizes a digital forensics internship project focused on detecting SQL and NoSQL injection attacks. The primary objective was to simulate real-world SQLi scenarios in a MySQL-backed Node.js web application and implement detection mechanisms using heuristic and machine learning (ML) approaches. One endpoint used safe prepared statements, while another remained vulnerable to simulate attacks. Tools like SQLMap demonstrated the effectiveness of both defenses. Logging mechanisms were added to capture attacker inputs, queries, and IPs for forensic purposes. Heuristic rules based on known patterns effectively blocked common attacks, while an ML model helped detect novel injection attempts.

The study was extended to MongoDB to examine NoSQL injection threats. A vulnerable route accepted unsanitized input in query objects, while another applied strict validation. Techniques like `$ne`, `$or`, and regex payloads were tested. Detection used both heuristic patterns and an ML model trained on JSON-based inputs. Though NoSQL lacks prepared statements, schema enforcement combined with adaptive detection proved effective.

Overall, the project offered hands-on experience with secure coding, attack simulation, and real-time logging. The findings highlight the importance of hybrid defenses for both SQL and NoSQL systems, emphasizing the need for continuous adaptation in web application security.

TABLE OF CONTENTS

1. Introduction	-----	5-6
2. Literature Review	-----	7-8
3. Methodology	-----	9-18
4. Results	-----	19-26
5. Case Studies/Practical Work	-----	27-32
6. Discussion	-----	33
7. Conclusion	-----	34
8. References	-----	35-36
9. Appendices	-----	37-38

LIST OF FIGURES AND TABLES

TABLES:

Table1: SQLMap Exploitation Steps for MySQL Injection

Table 2: Regex Patterns Used for Heuristic Detection of SQL Injection

Table 3: SQL injection payloads

Table 4: NoSQL Injection Steps with Malicious Payloads

Table 5: Regex Patterns for NoSQL Heuristic Detection

Table 6: NoSQL Injection Payloads Used in Testing

FIGURES:

Fig 1:SQLi Detection Flowchart (MySQL & NoSQL)

Fig 2:Confusion Matrix (MySQL)

INTRODUCTION

BACKGROUND

Digital forensics involves the investigation of digital devices to uncover and interpret electronic data. It plays a crucial role in cybersecurity, legal investigations, and corporate compliance. With the increasing prevalence of cybercrime, the need for skilled digital forensics professionals has never been greater.

OBJECTIVE

MySQL: The project aimed to implement forensic-based detection for SQL Injection in a MySQL-backed Node.js app by:

- Creating secure (prepared statements) and insecure routes.
- Simulating attacks using SQLMap.
- Logging attacker inputs, IPs, and query traces.
- Applying heuristic rules and a lightweight ML model for detection.

NoSQL (MongoDB): The extended objective was to study NoSQL injection in a MongoDB setup by:

- Building both vulnerable and protected endpoints.
- Testing attack patterns like `$ne`, `$or`, and regex-based payloads.
- Logging forensic data such as payloads and queries.
- Using heuristic signatures and a custom ML model to detect anomalies.
- Evaluating the effectiveness of hybrid (heuristic + ML) detection methods.

SCOPE

The report primarily focuses on SQL Injection detection using MySQL, detailing architecture, injection points, forensic logging, and detection methods. The NoSQL section, centered on MongoDB, explores schema-less vulnerabilities, query-based injections, and detection strategies using structured logging and ML models. Together, both phases contribute to a comparative analysis of SQL and NoSQL security.

LITERATURE REVIEW

MYSQL

SQL Injection (SQLi) remains one of the most prevalent and dangerous web-based attacks, making its detection a key focus in digital forensics and cybersecurity. Numerous studies have proposed advanced techniques combining machine learning, deep learning, and rule-based heuristics to identify and prevent SQLi attacks.

These are a few papers related to MySQL:

- An Efficient SQL Injection Detection with Hybrid CNN & Random Forest
- A Survey of SQL Injection Attacks, Their Methods, and Prevention Techniques
- A diligent survey of SQL injection attacks, detection and evaluation of mitigation techniques
- An Efficient SQL Injection Detection with Hybrid CNN & Random Forest
- SQL Injection Detection Using 2D-Convolutional Neural Networks (2D-CNN)
- Deep learning approaches to SQL injection detection: evaluating ANNs, CNNs, and RNNs
- Detection of SQL Injection Attacks Using Machine Learning Techniques

NOSQL

NoSQL Injection is an emerging threat due to the growing adoption of non-relational databases like MongoDB, CouchDB, and Firebase. Unlike traditional SQL-based systems, NoSQL databases often lack built-in query sanitization mechanisms and allow flexible query structures, making them susceptible to novel injection vectors, such as JSON tampering, operator injection (\$ne, \$gt, \$regex), and logic manipulation.

Recent research highlights the unique challenges posed by NoSQL injection attacks and proposes evolving detection methods, often adapted from SQLi mitigation strategies but modified for NoSQL's unstructured query format.

These are a few relevant papers and studies focusing on NoSQL injection detection:

- Securing NoSQL Databases Against Injection Attacks
- Detection and Prevention of NoSQL Injection Attacks in Web Applications
- Machine Learning-Based Anomaly Detection for NoSQL Injection
- A Comparative Study of SQL and NoSQL Injection Vulnerabilities in Web Applications
- Forensic Analysis of NoSQL Injection Attacks in Node.js Applications
- Using ML and Signature-Based Detection to Identify NoSQL Injections

METHODOLOGY

HOW THE HACKER GAINS ACCESS (MYSQL)

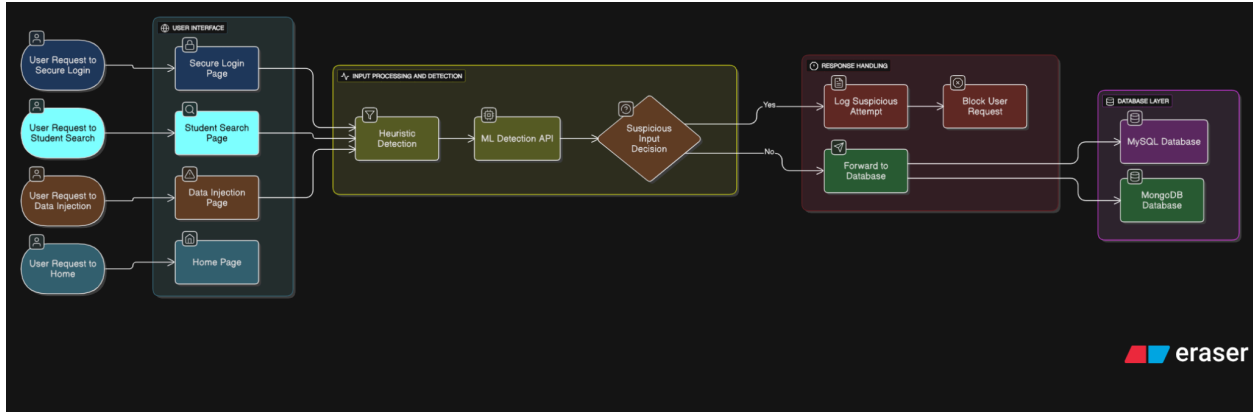
To demonstrate how an attacker could exploit SQL injection vulnerabilities, SQLMap, an open-source penetration testing tool, was used. SQLMap automates the process of detecting and exploiting SQL injection flaws and retrieving data from vulnerable databases.

Once a vulnerable input point is identified (such as the **term** parameter in **/student-search**), the attacker can use SQLMap. The table below outlines how specific commands were used to extract sensitive information from a vulnerable route

Step	Goal	SQLmap command	Explanation
1	Identify the DBMS and current DB	<u><code>python sqlmap/sqlmap.py -u "http://localhost:3000/student-search?term=abc" --batch --current-db</code></u>	Detects the database type (e.g., MySQL) and extracts the name of the active DB (login_demo).
2	List all tables in the database	<u><code>python sqlmap/sqlmap.py -u "http://localhost:3000/student-search?term=abc" --batch -D login_demo --tables</code></u>	Enumerates all tables under the discovered database by exploiting the same injection point.

(Table1:SQLMap Exploitation Steps for MySQL Injection)

ARCHITECTURE/FLOW DIAGRAM (MYSQL & NOSQL)



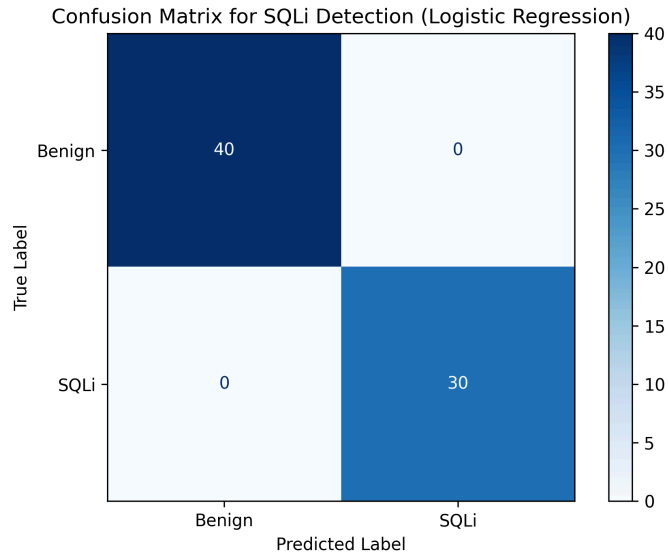
(Fig 1:SQLi Detection Flowchart)

The system follows a modular, multi-layered architecture to detect SQL Injection (SQLi) attacks in real-time.As illustrated in the diagram:

- The user interacts with a web interface built in Node.js, accessing pages like secure login, student search, and data injection.
- Inputs are passed through both **heuristic rules** and a **Flask-based machine learning (ML) API** for detection.
- If the input is deemed suspicious, it is logged and blocked; otherwise, it is forwarded to the **MySQL/NOSQL** database.
-

TOOLS AND TECHNOLOGIES (MYSQL & NOSQL)

- **Backend:** Node.js with Express.js for routing and middleware.
- **Database (MySQL):** MySQL and NoSQL used to simulate traditional relational database injection.
- **Detection:**
 - **Heuristic Layer:** Regex-based filtering for typical SQLi payloads.
 - **ML Layer:** Flask API serving a Logistic Regression model trained on vectorized input (TF-IDF, using **scikit-learn**).



(Fig 2:Confusion Matrix for MySQL)

- **Logging:** Suspicious inputs are saved using Node.js `fs` module for forensic review.

PROCEDURE (MYSQL)

1. User Input Collection

- Inputs are taken from the web app via login and search forms.

2. Heuristic Detection

- Predefined regular expressions (e.g., presence of ' `OR` ' `1` '=' `1`, `--`, `SELECT`, `DROP`) are used to flag common SQLi patterns.

Pattern Description	Regex Pattern	Examples Detected
SQL meta-characters and comment markers	<code>`(%27)</code>	<code>(')</code>

Logical conditions (AND/OR with comparisons)	\b(OR	AND)\b.+(=
Common SQL keywords for injection	\b(UNION	SELECT
Time-based injection (MySQL sleep)	\bSLEEP\s*\b(	SLEEP(5)
Time-based injection (SQL Server delay)	\bWAITFOR\b s+DELAY\b	WAITFOR DELAY '00:00:10'
Tautology-based injection	["']\s*OR\s*["']1["']=["']1	' OR '1'='1, " OR "1"="1"

(Table 2:Regex Patterns Used for Heuristic Detection of SQL Injection)

3. ML-based Analysis

- Input is also passed to a Flask API that returns a classification (**malicious** or **safe**) using a trained Logistic Regression model.
- A dataset with 350 samples was used.

4. Decision Logic

- If **both layers** label the input as safe, the request proceeds to the **MySQL** database.
- If either layer detects malicious behavior, the input is **logged and blocked**.

5. Logging for Digital Forensics

- All flagged inputs are logged with metadata including timestamp, route (**/login**, **/student-search**), and IP address.

6. Testing and Validation

- SQL injection payloads (e.g., ' ; DROP TABLE users --) are manually tested to validate whether the system detects and blocks them correctly.
- The below table explains them:

Category	Injected Payload	Effect
Tautology	x%' OR '1'='1	Bypasses
Insert-Users	x%'; INSERT INTO users (id,username,password,role) VALUES (8,'hacker2','4321','admin') --	Inserts fake admin to users table
Insert-Students	x%'; INSERT INTO students (id,name,department,marks) VALUES (3,'Charlie','MECH',78) --	Adds a new student record
Update-Boost marks	x%'; UPDATE students SET marks = 100 WHERE name="Bob" --	Modifies Bob's marks to 100
Update-Zero marks	x%'; UPDATE students SET marks = 0 WHERE name="Charlie" --	Modifies Charlie's marks to 0
Delete	x%'; DELETE FROM students WHERE name="Charlie" --	Deletes Charlie's record
Drop Tables	x%'; DROP TABLE students --	Deletes entire students table

(Table3:SQL injection payloads)

DATA COLLECTION (MYSQL)

- **SQLi Dataset:** `sqli_tiny_dataset.csv`, containing labeled examples of safe vs. malicious queries.
- **Manual Testing:** Realistic SQLi payloads crafted and tested via the app interface.
- **Logs:** Real-time user activity logs collected during testing for forensic analysis.
- **Database Response:** Query outcomes on MySQL observed to confirm if attack was executed or blocked.

ANALYSIS METHOD (MYSQL)

- **Heuristic Pattern Matching**
 - Regular expressions flag inputs containing suspicious tokens, keywords, or known SQLi signatures.
- **Machine Learning Classification**
 - Input is vectorized using **TF-IDF**, passed to a **Logistic Regression** classifier deployed as a Flask API.
 - API returns a JSON response with the predicted label (`safe` or `malicious`).
- **Log Review**
 - Logged inputs are manually examined to evaluate detection quality and fine-tune rules if necessary.
- **Model Evaluation**

- Metrics like **accuracy**, **precision**, **recall**, and **F1-score** are calculated using `classification_report()` during model training.

HOW THE HACKER GAINS ACCESS (NOSQL)

To simulate how attackers exploit NoSQL injection vulnerabilities, a custom web route was deliberately left vulnerable in a Node.js + Express app connected to MongoDB. Unlike SQL injections that rely on breaking structured queries, NoSQL injections manipulate JSON-based query structures using operators like `$ne`, `$or`, and regular expressions.

The attacker exploits endpoints where raw user input is directly passed into MongoDB queries. Using crafted payloads, they can bypass authentication, extract data, or manipulate existing records.

Step	Goal	Payload / Input Example	Explanation
1	Bypass login	<code>{"username": {"\$ne": null}}</code>	Returns all users, bypassing auth.
2	Extract all student data	<code>{"name": {"\$regex": ".*"}}</code>	Returns all student records.
3	Tamper with data	<code>{"\$where": "this.marks > 50"}</code>	Executes logic-based filter (if enabled).
4	Trigger regex DoS or slow query	<code>{"name": {"\$regex": ".*a.*a.*a.*"}}</code>	Abuses regex engine to cause delay.

(Table 4: NoSQL Injection Steps with Malicious Payloads)

PROCEDURE (NOSQL)

- 1. User Input Collection** – User-provided inputs are collected from endpoints like login and student search in the Node.js app.

- 2. Heuristic Detection** – Inputs are matched against a set of known NoSQL attack patterns based on JSON operators, logic conditions, and regex usage.

Pattern Type	Regex Pattern	Examples Detected
MongoDB Operator Injection	"\$ne"	"\$gt"
Logic Bypass	"\$or" or "\$where"	{"\$or": [{"name": "Alice"}, {}]}
Tautologies in JSON	.*\s*\{\s*"ne"\s*:\s*null\s*\}	{"username": {"ne": null}}
Regex Abuse	"\$regex":\s*".*"	{"name": {"regex": ".*"}}

(Table 5: Regex Patterns for NoSQL Heuristic Detection)

- 3. ML-based Analysis** – Inputs are also sent to a Flask-based ML API which classifies them using a trained Logistic Regression model on vectorized text.

- Dataset: 300 labeled NoSQL query samples
- Features: TF-IDF vectors trained on JSON-like text
- Model Output: {"result": "safe"} or {"result": "malicious"}

4. Decision Logic:

- If **both heuristic and ML** layers pass → input is forwarded to MongoDB.
- If **any layer flags malicious behavior** → input is blocked and logged.

- 5. Logging for Digital Forensics** – Each suspicious request is logged with:

- Timestamp
- Endpoint (/login, /student-search)
- IP address

- Raw input / query
- ML prediction result

6. Testing and Validation – Real-world NoSQL injection patterns were manually tested, including:

Category	Injected Payload	Effect
Bypass login	{"username": {"\$ne": null}}	Logs in attacker
Read all students	{"name": {"\$regex": ".*"}}	Lists all records
Regex DoS	{"name": {"\$regex": ".*a.*a.*a.*a.*"}}	Slows down DB
Update document	Injected via req.body in a POST route	Alters or inserts data
Drop Collection	Indirect attempt using \$where (if JS enabled)	Destructive manipulation

(Table 6: NoSQL Injection Payloads Used in Testing)

DATA COLLECTION (NOSQL)

- **NoSQL Injection Dataset:** **nosqli_dataset.csv** with labeled samples (safe/malicious).
- **Manual Input Testing:** Crafted payloads tested via app UI and Postman.
- **System Logs:** Forensic logs saved to **/logs/nosqli_logs.txt**.
- **Query Outcomes:** MongoDB responses monitored to verify successful blocking or exploitation.

ANALYSIS METHOD (NOSQL)

- **Heuristic Pattern Matching:**
 - Matches input strings against known NoSQL attack patterns.
 - **Machine Learning Classification:**
 - Text input → TF-IDF vector → Logistic Regression model → Prediction label.
 - **Log Review:**
 - Logs reviewed post-testing to analyze false positives/negatives and fine-tune regex patterns.
 - **Model Evaluation:**
 - Accuracy, Precision, Recall, F1-score computed using `classification_report()` in scikit-learn.
 - 5-fold cross-validation used during training phase.
-

RESULTS

MYSQL

SQLMap:

When the commands in table 1 are run :

Command 1:

Gives the DBMS and database name

```
---
[21:38:43] [INFO] the back-end DBMS is MySQL
web application technology: Node.js (Express), Express
back-end DBMS: MySQL 5 (MariaDB fork)
[21:38:43] [INFO] fetching current database
current database: 'login_demo'
[21:38:43] [INFO] fetched data logged to text files under 'C:\Users\varsh\AppData\Local\sqlmap\output\localhost'

[*] ending @ 21:38:43 /2025-07-22/

PS C:\Users\varsh\sql_injection> █
```

Command 2:

Gives the tables in the database

```
back-end DBMS: MySQL 5 (MariaDB fork)
[21:39:39] [INFO] fetching tables for database: 'login_demo'
Database: login_demo
[2 tables]
+-----+
| students |
| users    |
+-----+

[21:39:39] [INFO] fetched data logged to text files under 'C:\Users\varsh\AppData\Local\sqlmap\output\localhost'

[*] ending @ 21:39:39 /2025-07-22/
```

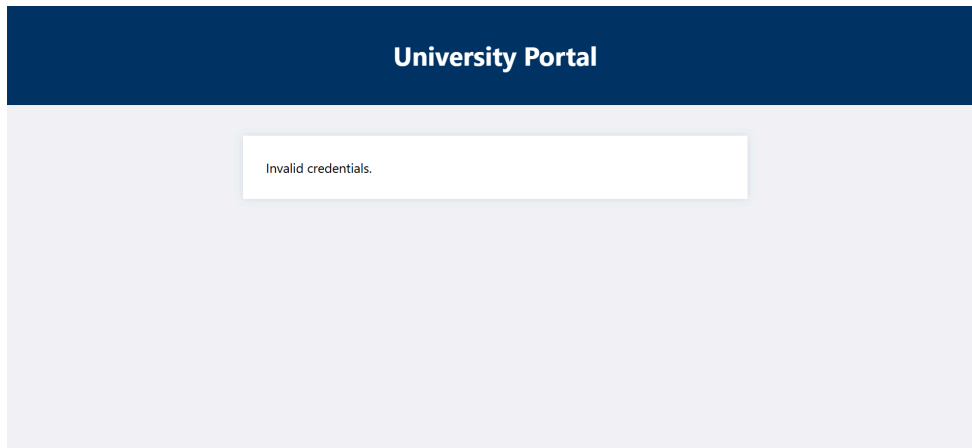
WEBSITE

University Portal

Username:

Password:

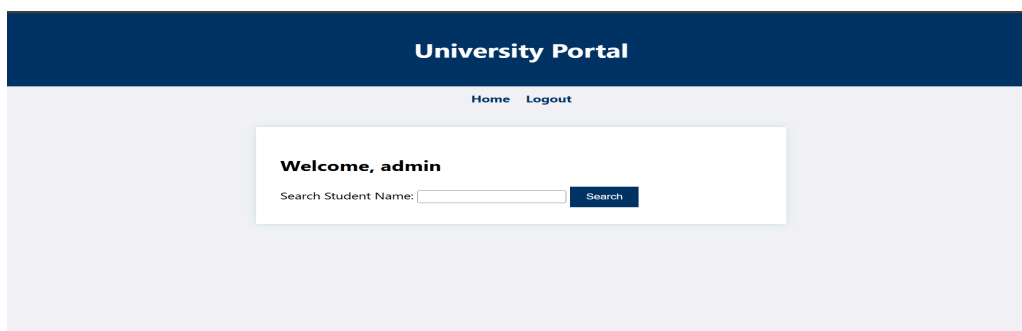
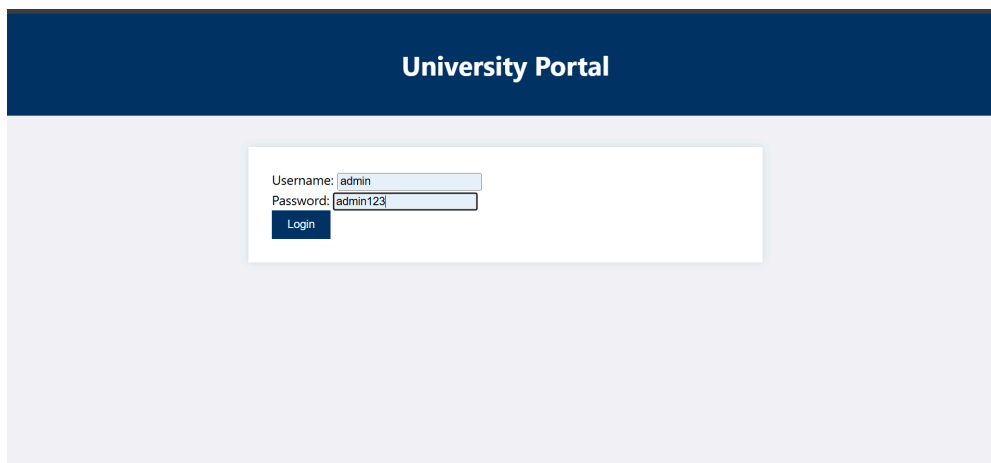
Login



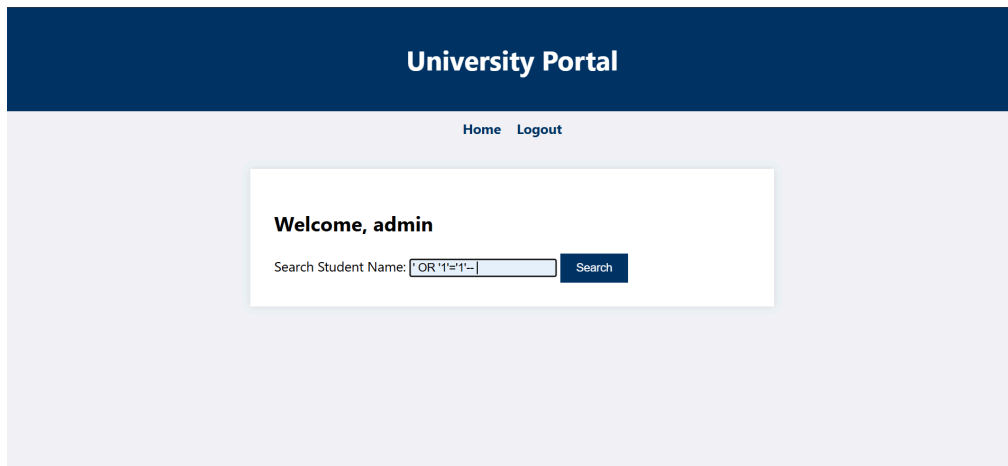
Logs.txt:

```
[2025-08-03T05:53:48.093Z] [LOGIN] IP: ::1 | Username: ' OR '1'='1'-- | Password:
```

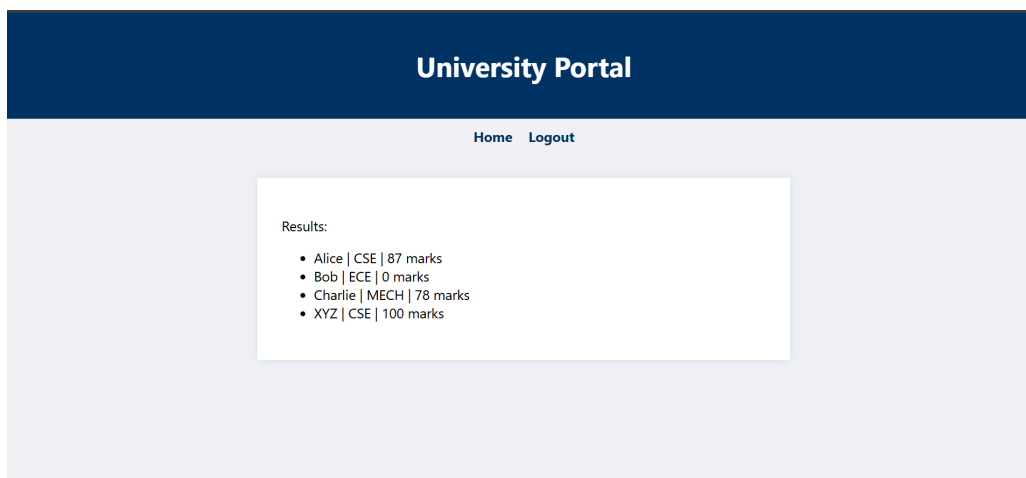
This is normal login and it logs in



Now when i try to perform injection in the vulnerable search part,the following happens:



The screenshot shows the 'University Portal' header. Below it are links for 'Home' and 'Logout'. A central white box contains the text 'Welcome, admin'. Below this is a search form with the label 'Search Student Name:'. The input field contains the SQL injection payload 'OR '1'='1--'. To the right of the input field is a blue 'Search' button.



The screenshot shows the 'University Portal' header. Below it are links for 'Home' and 'Logout'. A central white box displays the search results. It starts with the heading 'Results:' followed by a bulleted list of student records.

- Alice | CSE | 87 marks
- Bob | ECE | 0 marks
- Charlie | MECH | 78 marks
- XYZ | CSE | 100 marks

This is protected using Heuristics and A ML model that uses Logistic Regression which blocks and logs all suspicious inputs in login as well as search

University Portal

Username:

' OR '1'='1'--

Password:

Login

University Portal

Suspicious input detected. Access denied.

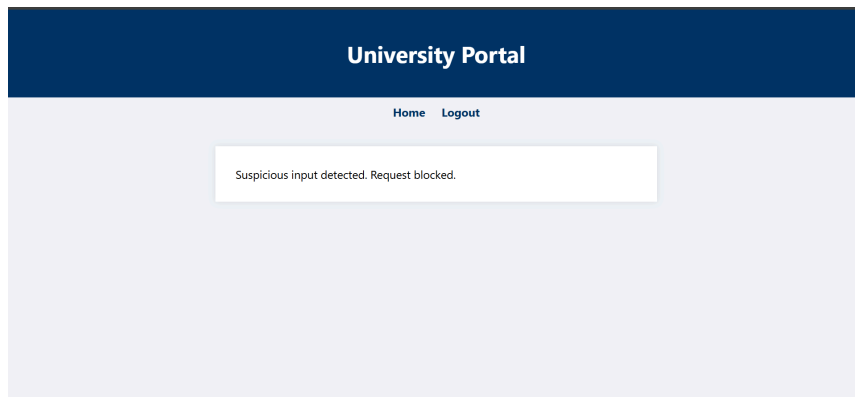
Command:
x%' ; INSERT INTO users (id,username, password, role) VALUES
(5,'student2','7890','student') --

University Portal

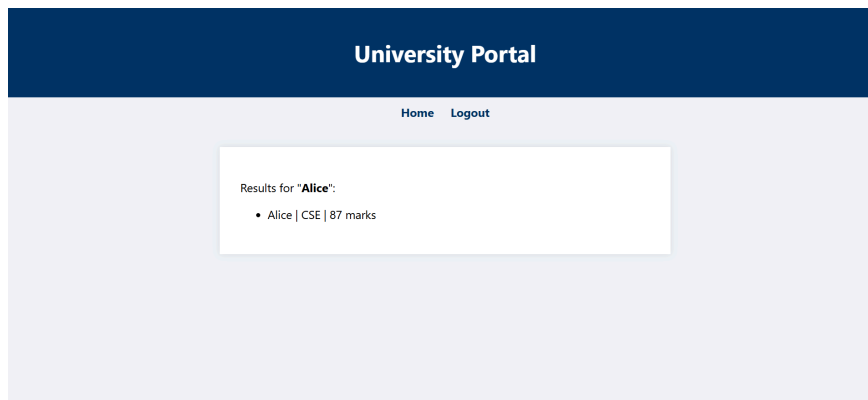
[Home](#) [Logout](#)

Welcome, admin

Search Student Name: x%' ; INSERT INTO users (Search

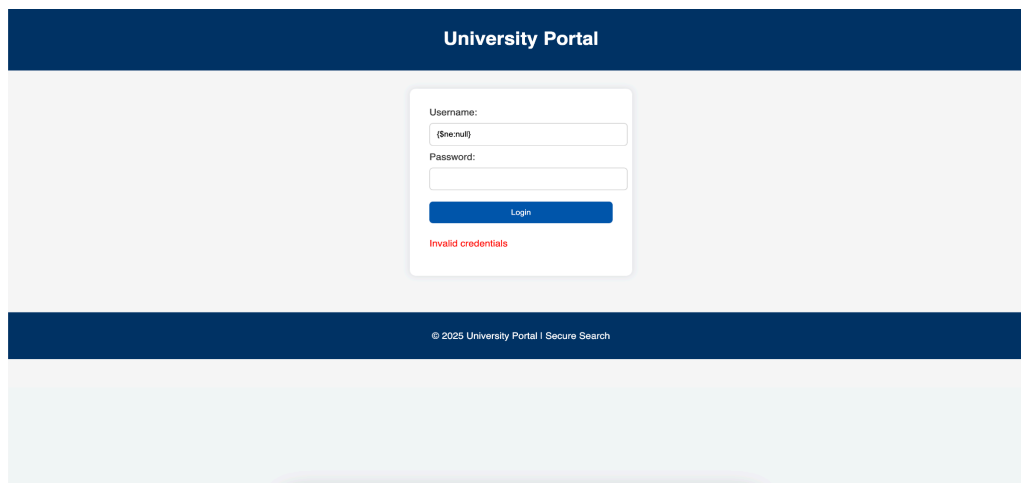


search:Alice

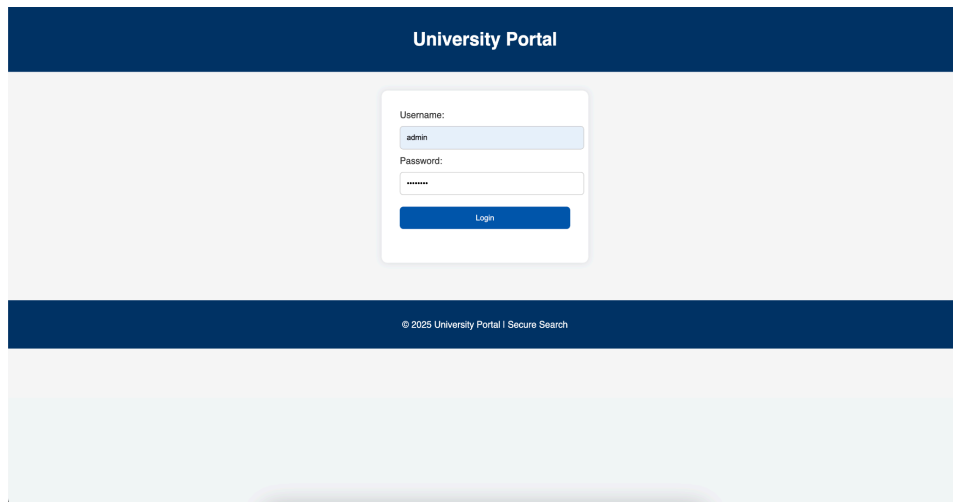


NOSQL

WEBSITE

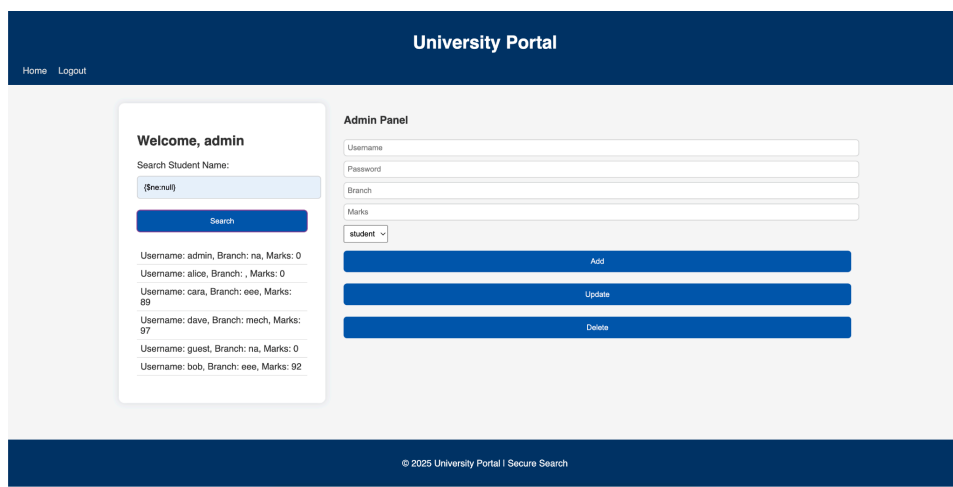


NORMAL LOGIN



The image shows a web application titled "University Portal". It features a login form in the center with two input fields: "Username:" containing the text "admin" and "Password:" containing seven asterisks. Below these fields is a blue "Login" button. The form is set against a light gray background. At the bottom of the page, there is a dark blue footer bar with the text "© 2025 University Portal | Secure Search".

Performing injection in Vulnerable Search



The image shows the "University Portal" Admin Panel. The top navigation bar includes "Home" and "Logout" links. The main content area is divided into two sections. On the left, a "Welcome, admin" message is followed by a "Search Student Name:" section with a text input field containing "\${ne.null}" and a blue "Search" button. Below this, a list of student records is displayed: "Username: admin, Branch: na, Marks: 0", "Username: alice, Branch: , Marks: 0", "Username: cara, Branch: eee, Marks: 89", "Username: dave, Branch: mech, Marks: 97", "Username: guest, Branch: na, Marks: 0", and "Username: bob, Branch: eee, Marks: 92". On the right, the "Admin Panel" contains input fields for "Username", "Password", "Branch", and "Marks", along with a "student" dropdown menu. Below these fields are three blue buttons labeled "Add", "Update", and "Delete". The footer bar at the bottom contains the text "© 2025 University Portal | Secure Search".

Protected using Heuristics and a ML model that uses Logistic Regression

HomeLogout

University Portal

Welcome, admin

Search Student Name:

([null])

Search

Admin Panel

Username

Password

Branch

Marks

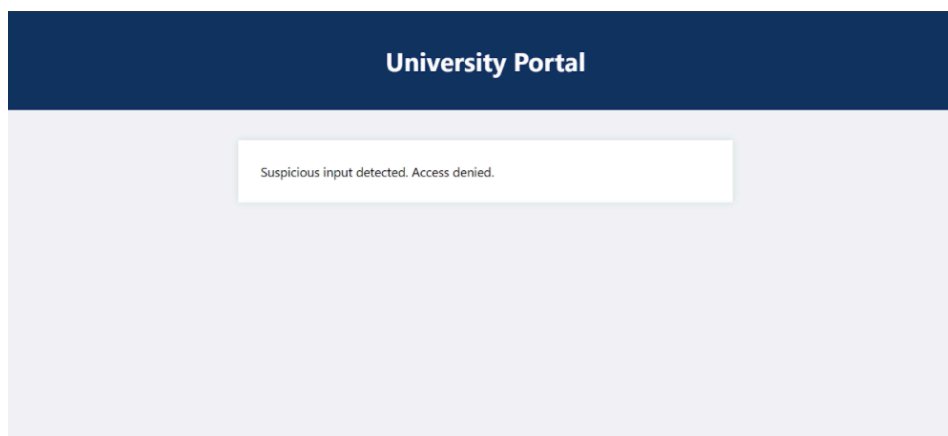
student

Add

Update

Delete

© 2025 University Portal | Secure Search



DATA MANIPULATION

HomeLogout

localhost:3000 says
User added

OK

Welcome, admin

Search Student Name:

Enter name or query...

Search

Admin Panel

Alice

null

Branch

0

student

Add

Update

Delete

© 2025 University Portal | Secure Search

University Portal

[Home](#) [Logout](#)

Welcome, admin

Search Student Name:

Alice

Search

Username: alice, Branch: , Marks: 0

Admin Panel

Username

Password

Branch

Marks

student

Add

Update

Delete

© 2025 University Portal | Secure Search

LOGS.TXT

```
project > server > E logs.txt
18 2025-08-03T03:36:10.558Z - LOGIN: admin (admin)
19 2025-08-03T03:36:10.558Z - MALICIOUS SEARCH by admin: {$ne:null}
20 2025-08-03T03:36:19.190Z - UPDATE: Admin admin updated alice
21 2025-08-03T03:36:30.989Z - DELETE: Admin admin deleted bob
22 2025-08-03T03:37:35.604Z - UPDATE: Admin admin updated alice
23 2025-08-03T03:39:30.605Z - LOGIN: alice (student)
24 2025-08-03T03:39:34.531Z - MALICIOUS SEARCH by alice: {$ne:null}
25 2025-08-03T03:40:38.210Z - LOGIN: admin (admin)
26 2025-08-03T03:40:48.088Z - ADD: Admin admin added user bob
27 2025-08-03T03:41:39.282Z - FAILED LOGIN: alice
28 2025-08-03T03:41:46.981Z - LOGIN: alice (student)
29 2025-08-03T03:41:50.635Z - LOGIN: admin (admin)
30 2025-08-03T03:53:46.100Z - UPDATE: Admin admin updated alice
31 2025-08-04T04:43:16.723Z - LOGIN: admin (admin)
32 2025-08-04T04:43:23.683Z - MALICIOUS SEARCH by admin: {$ne:null}
33 2025-08-04T04:43:36.060Z - LOGIN: admin (admin)
34 2025-08-04T04:43:59.653Z - ADD: Admin admin added user Alice
35 2025-08-04T04:50:46.165Z - LOGIN: admin (admin)
36 2025-08-04T04:51:14.766Z - FAILED LOGIN: {$ne:null}
37 2025-08-04T05:14:23.528Z - LOGIN: admin (admin)
38
```

CASE STUDY/PRACTICAL WORK

Case Study 1: Injection via Vulnerable Search Endpoint

MYSQL

- Objective: To test if search functionality is vulnerable to SQL injection.
- Payload Used:
`x%' ; INSERT INTO students (id,name, department, marks) VALUES (4,'pqr','CSE',100) --`
- Database before:

Extra options

						id	name	department	marks	
☐		Edit		Copy		Delete	1	Alice	CSE	87
☐		Edit		Copy		Delete	2	Bob	ECE	0
☐		Edit		Copy		Delete	3	Charlie	MECH	78
☐		Edit		Copy		Delete	4	XYZ	CSE	100

Database After:

← T →					id	name	department	marks		
☐		Edit		Copy		Delete	1	Alice	CSE	87
☐		Edit		Copy		Delete	2	Bob	ECE	0
☐		Edit		Copy		Delete	3	Charlie	MECH	78
☐		Edit		Copy		Delete	4	XYZ	CSE	100
☐		Edit		Copy		Delete	5	pqr	CSE	100

Logs.txt

```
[2025-08-03T06:01:25.384Z] [SEARCH] IP: ::1 | Input: x%' ; INSERT INTO students (id,name, department, marks) VALUES (5,'pqr','CSE',100) --
SQL: x%' ; INSERT INTO students (id,name, department, marks) VALUES (5,'pqr','CSE',100) --
DATA MANIPULATED-----
Added:
{
  "id": 5,
  "name": "pqr",
  "department": "CSE",
  "marks": 100
}
```

NOSQL

- Objective: To test if search functionality is vulnerable to SQL injection.

The screenshot displays the 'University Portal' Admin Panel. On the left, a 'Welcome, admin' message is followed by a search bar labeled 'Search Student Name:' with a placeholder '(line: null)'. Below the search bar is a blue 'Search' button. To the right of the search bar, a list of users is shown in a table-like format with columns for Username, Branch, and Marks. The users listed are admin, alice, cara, dave, and guest. The 'admin' user has 0 marks, while the others have 89 marks. On the right side of the panel, there is an 'Admin Panel' section with four input fields for Username, Password, Branch, and Marks, followed by a dropdown menu currently set to 'student'. Below these fields are four blue buttons: 'Add', 'Update', and 'Delete'.

Compass

My Queries

connections (8)

Search connections

localhost:27017

admin

config

local

log_status

project

users

university

university_journal

localhost:27017 > project > users

Documents

Aggregations

Schema

Indexes

Validation

Type a query: (field: "value") => **66a6c4e_mercy** ✓

Explain

Reset

Fetch

Options

25 1 of 6

ADD DATA

EXPORT DATA

UPDATE

DELETE

```

{
  "$if": { "$eq": { "$type": "66a6d0d0b132d3c8a12b4e7a" } },
  username: "Admin",
  password: "Admin123",
  role: "admin",
  branch: "Test",
  marks: 0
  ...x: 0
},
{
  "$if": { "$eq": { "$type": "66a6d0d0b132d3c8a12b4e7a" } },
  username: "Ali",
  password: "Ali123",
  role: "student",
  branch: "Test",
  marks: 0
  ...x: 0
},
{
  "$if": { "$eq": { "$type": "66a6d0d0b132d3c8a12b4e7a" } },
  username: "Ayaz",
  password: "Ayaz123",
  role: "teacher",
  branch: "Test",
  marks: 0
  ...x: 0
},
{
  "$if": { "$eq": { "$type": "66a6d0d0b132d3c8a12b4e7a" } },
  username: "Ayaz",
  password: "Ayaz123",
  role: "student",
  branch: "Test",
  marks: 0
  ...x: 0
},
{
  "$if": { "$eq": { "$type": "66a6d0d0b132d3c8a12b4e7a" } },
  username: "Ayaz",
  password: "Ayaz123",
  role: "teacher",
  branch: "Test",
  marks: 0
  ...x: 0
},
{
  "$if": { "$eq": { "$type": "66a6d0d0b132d3c8a12b4e7a" } },
  username: "Ayaz",
  password: "Ayaz123",
  role: "student",
  branch: "Test",
  marks: 0
  ...x: 0
}

```

- Logs Entry:

```
project> server -F logs.txt
23 2025-08-03T03:39:30.605Z - LOGIN: alice (student)
24 2025-08-03T03:39:34.531Z - MALICIOUS SEARCH by alice: {$ne:null}
25 2025-08-03T03:40:38.216Z - LOGIN: admin (admin)
26 2025-08-03T03:40:48.088Z - ADD: Admin admin added user bob
27 2025-08-03T03:41:39.282Z - FAILED LOGIN: alice
28 2025-08-03T03:41:46.981Z - LOGIN: alice (student)
29 2025-08-03T20:53:08.635Z - LOGIN: admin (admin)
30 2025-08-03T20:53:46.100Z - UPDATE: Admin admin updated alice
31 2025-08-04T04:43:16.723Z - LOGIN: admin (admin)
32 2025-08-04T04:43:23.683Z - MALICIOUS SEARCH by admin: {$ne:null}
33 2025-08-04T04:43:36.060Z - LOGIN: admin (admin)
34 2025-08-04T04:43:59.653Z - ADD: Admin admin added user Alice
35 2025-08-04T04:50:46.165Z - LOGIN: admin (admin)
36 2025-08-04T04:51:14.766Z - FAILED LOGIN: {$ne:null}
37 2025-08-04T05:14:23.528Z - LOGIN: admin (admin)
38 2025-08-04T05:28:04.932Z - LOGIN: admin (admin)
39 2025-08-04T05:34:31.950Z - DELETE: Admin admin deleted Bob
40 2025-08-04T05:34:31.950Z - DELETE: Admin admin updated Alice2025-08-04T06:21:07.529Z - LOGIN: admin (admin)
41 2025-08-04T06:21:12.672Z - MALICIOUS SEARCH by admin: {$ne:null}
42
```

Case Study 2: Deletion via Injection

MYSQL

- Payload:

`x%'; DELETE FROM students WHERE name='Charlie' --`

Database before

← T →						id	name	department	marks
☐	 Edit	 Copy	 Delete	1	Alice	CSE	87		
☐	 Edit	 Copy	 Delete	2	Bob	ECE	0		
☐	 Edit	 Copy	 Delete	3	Charlie	MECH	78		
☐	 Edit	 Copy	 Delete	4	XYZ	CSE	100		
☐	 Edit	 Copy	 Delete	5	pqr	CSE	100		

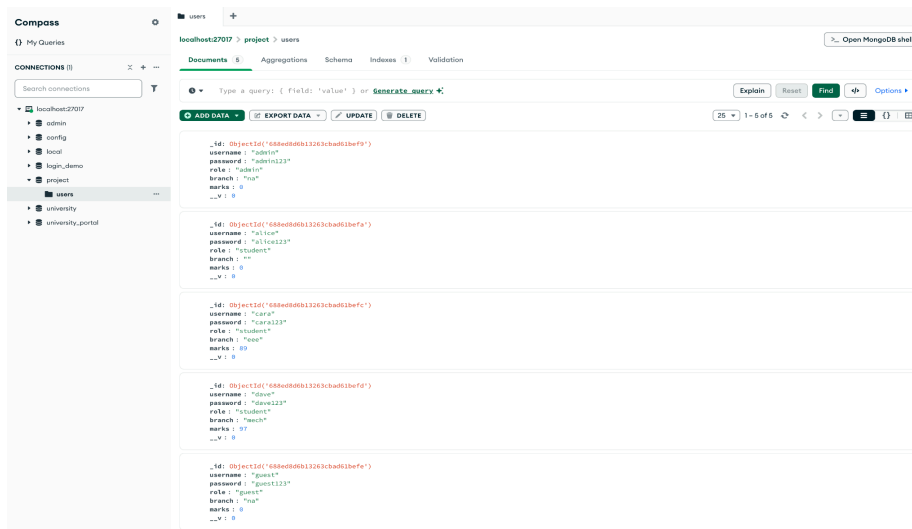
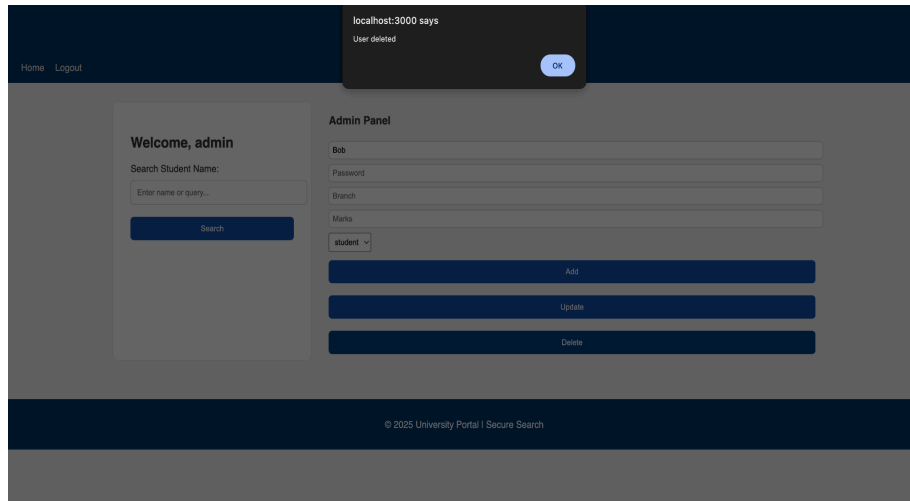
Database after

←T→					id	name	department	marks
☐	 Edit	 Copy	 Delete	1	Alice	CSE	87	
☐	 Edit	 Copy	 Delete	2	Bob	ECE	0	
☐	 Edit	 Copy	 Delete	4	XYZ	CSE	100	
☐	 Edit	 Copy	 Delete	5	pqr	CSE	100	

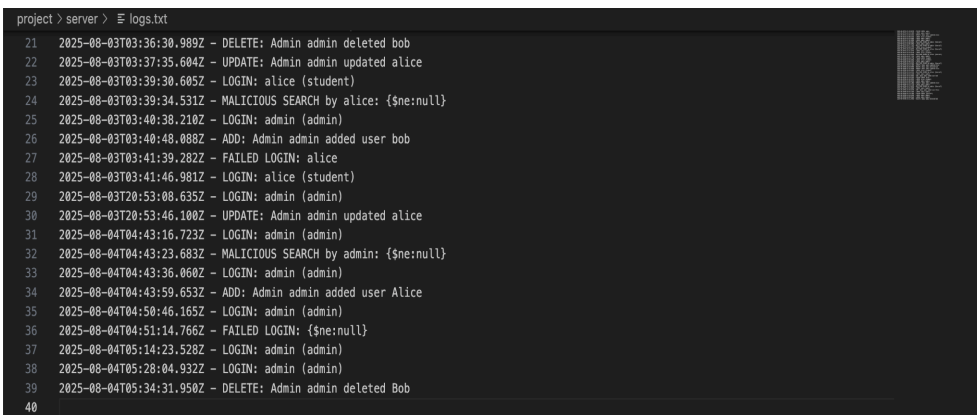
Logs.txt

```
[2025-08-03T06:51:19.893Z] [SEARCH] IP: ::1 | Input: x%'; DELETE FROM students WHERE name='Charlie' --
SQL: x%'; DELETE FROM students WHERE name='Charlie' --
DATA MANIPULATED-----
Deleted:
[
  {
    "id": 3,
    "name": "Charlie",
    "department": "MECH",
    "marks": 78
  }
]
```

NOSQL



- Logs Entry:



Case Study 3: Account Updation via Injection

MYSQL

- Payload:
`x%' ; UPDATE students SET marks = 100 WHERE name = "Bob" --`
- Outcome: Marks updated to 100

Database Before:

				id	name	department	marks
☐				1	Alice	CSE	87
☐				2	Bob	ECE	0
☐				4	XYZ	CSE	100
☐				5	pqr	CSE	100

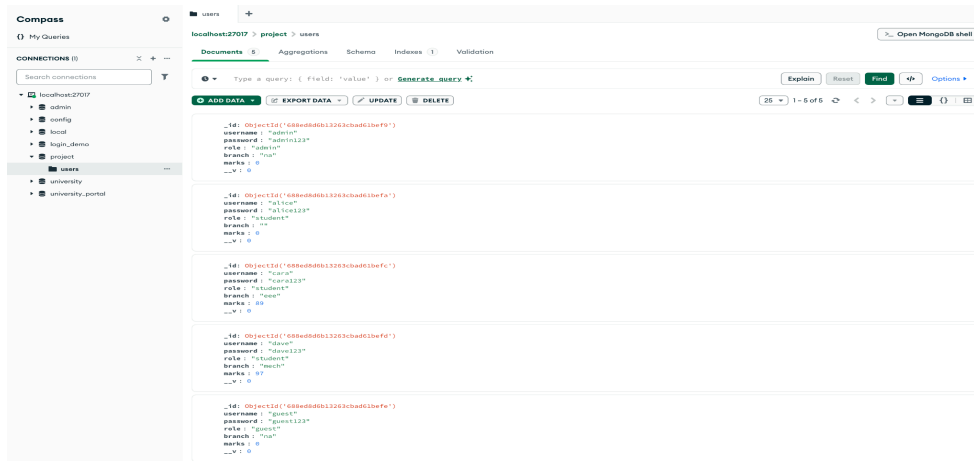
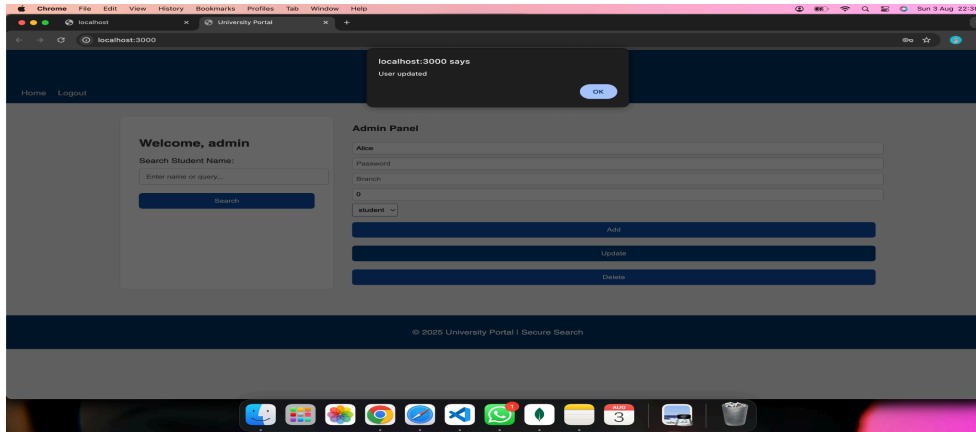
Database after:

				id	name	department	marks
☐				1	Alice	CSE	87
☐				2	Bob	ECE	100
☐				4	XYZ	CSE	100
☐				5	pqr	CSE	100

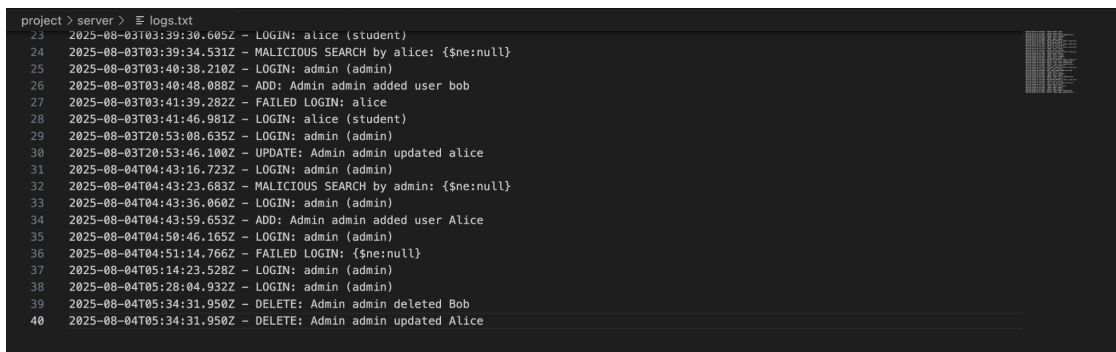
- Log Entry:

```
[2025-08-03T06:55:19.496Z] [SEARCH] IP: ::1 | Input: x%' ; UPDATE students SET marks = 100 WHERE name = "Bob" --
SQL: x%' ; UPDATE students SET marks = 100 WHERE name = "Bob" --
DATA MANIPULATED-----
Modified:
[
  {
    "before": {
      "id": 2,
      "name": "Bob",
      "department": "ECE",
      "marks": 0
    },
    "after": {
      "id": 2,
      "name": "Bob",
      "department": "ECE",
      "marks": 100
    }
  }
]
```

NOSQL



• Log Entry:



DISCUSSION

This project successfully developed a secure and educational web application that demonstrates both vulnerable and protected SQL injection (SQLi) scenarios. Using a MySQL-backed Node.js setup, the system features dual endpoints—one secured with prepared statements and another intentionally left open to simulate attacks. A heuristic detection system was integrated to identify and log malicious inputs in real time. Results from manual testing and tools like SQLMap confirmed that prepared statements completely blocked all injection attempts on the secure route. The vulnerable endpoint, as expected, allowed successful injections; however, the heuristic detection system accurately flagged common SQLi types such as tautologies, union-based queries, and manipulation attempts. Each flagged request was logged with attacker IP, payload, and reconstructed query for forensic analysis. Notably, the system maintained good usability. Legitimate users experienced no disruption, as only malformed or suspicious inputs were flagged. This balance between security and user experience shows that lightweight heuristic detection can coexist with standard web functionality when implemented carefully. However, the system does have limitations. Heuristic rules may fail to detect highly obfuscated or zero-day payloads. There is also a risk of false positives when valid inputs resemble malicious patterns, and reliance on IP-based logging may be unreliable in cases of proxy or VPN usage. For improvement, a machine learning module can be integrated to adaptively detect new or evolving attack patterns. ML models trained on labeled attack data can help reduce false positives and uncover novel injection variants. Additionally, extending the study to include NoSQL injection detection—specifically targeting MongoDB—will provide a broader understanding of modern data-layer vulnerabilities. Overall, this project demonstrates the practical value of combining secure coding practices with real-time forensic logging and rule-based detection. It offers a useful model for both teaching and testing web application security in controlled environments.

CONCLUSION

This project successfully demonstrated a practical approach to detecting and preventing SQL Injection (SQLi) attacks using heuristic rules, manual testing, and secure coding practices. A dual-route Node.js web application was developed — one showcasing a secure login system using prepared statements, and another deliberately left vulnerable to highlight common SQLi threats.

By integrating heuristic detection techniques, the system was able to log suspicious queries and patterns in real-time without compromising the functionality for normal users. Manual testing across multiple injection types — including tautologies, union-based attacks, comment injections, and data manipulation — revealed the importance of both input validation and behavioral monitoring.

The results showed that:

- Prepared statements completely mitigated SQLi attacks.
- Vulnerable endpoints were exploitable but were accurately detected and logged.
- Heuristic logging provided insight into attacker behavior, useful for digital forensics.

REFERENCES

MYSQL

- El Sayed, H., ElShafey, D., & El-Sayed, M. (2021). *An efficient SQL injection detection with hybrid CNN & Random Forest*. International Journal of Advanced Computer Science and Applications, 12(3), 120–127. <https://doi.org/10.14569/IJACSA.2021.0120316>
- Siddiqui, M., & Naaz, M. (2020). *Detection of SQL injection attacks using machine learning techniques*. International Journal of Computer Applications, 176(28), 1–5. <https://doi.org/10.5120/ijca2020920131>
- Srivastava, A., & Lal, S. (2019). *SQL injection detection using 2D-Convolutional Neural Networks (2D-CNN)*. International Journal of Innovative Technology and Exploring Engineering (IJITEE), 8(12), 3447–3451.
- Halfond, W. G. J., Viegas, J., & Orso, A. (2006). *A classification of SQL-injection attacks and countermeasures*. Proceedings of the IEEE International Symposium on Secure Software Engineering, 1–12.
- Shar, L. K., Tan, H. B. K., & Briand, L. C. (2013). *Mining SQL injection and cross-site scripting vulnerabilities using hybrid program analysis*. Proceedings of the 2013 International Conference on Software Engineering, 642–651. <https://doi.org/10.1109/ICSE.2013.6606594>
- SQLMap Project. (n.d.). *Automated SQL injection and database takeover tool*. Retrieved from <https://sqlmap.org>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, E. (2011). *Scikit-learn: Machine learning in Python*. Journal of Machine Learning Research, 12, 2825–2830. <https://scikit-learn.org>

NOSQL

- T. C. S, C. Sai Kapuluru and B. B, "Mitigating NoSQL Injection Vulnerabilities: Techniques, Examples, and Best Practices," 2025 International Conference on Emerging Systems and Intelligent Computing (ESIC), Bhubaneswar, India, 2025, pp.
<https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=10962645&isnumber=10962563>
 - O. Abahussain and A. Alqaddoumi, "DBMS, NoSQL and Securing Data: the relationship and the recommendation," 2020 International Conference on Innovation and Intelligence for Informatics, Computing and Technologies (3ICT), Sakheer, Bahrain, 2020, pp.
<https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=9311958&isnumber=9311940>
 - J. Kumar and V. Garg, "Security analysis of unstructured data in NOSQL MongoDB database," 2017 International Conference on Computing and Communication Technologies for Smart Nation (IC3TSN), Gurgaon, India, 2017, pp.
<https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=8284495&isnumber=8284439>
 - H. Matholia and O. M. Adedayo, "Investigating NoSQL Injection Attacks on MongoDB Web Applications," 2025 13th International Symposium on Digital Forensics and Security (ISDFS), Boston, MA, USA, 2025, pp.
<https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=11011969&isnumber=11011907>
 - V. Sachdeva and S. Gupta, "Basic NOSQL Injection Analysis And Detection On MongoDB," 2018 International Conference on Advanced Computation and Telecommunication (ICACAT), Bhopal, India, 2018, pp.
<https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=8933707&isnumber=8933531>
-

APPENDICES

MYSQL

- Appendix A: Sample Code Snippets
 - Heuristic rule (Regex) snippet
 - Node.js route handler example
 - ML model Flask API code
- Appendix B: Dataset Sample
 - Few rows from [sqli_tiny_dataset.csv](#)
- Appendix C: Logs.txt sample
 - Example log with timestamp, input, route, IP
- Appendix D: SQLMap command list (from Table 1)

NOSQL

- Appendix A: Sample Code Snippets
 - Heuristic rule (Regex) snippet
 - Node.js route handler example
 - ML model Flask API code
- Appendix B: Dataset Sample
 - Sample labeled input examples used to train the NoSQL injection classifier

- Appendix C: Logs.txt sample
 - Example log with timestamp, input, route, IP
 - Appendix D: NoSQL Injection Payload List
-